

Python 3D: Ursina Engine

Krystal S. Pérez Rosado - R00642760

Ezequiel D. Figueroa Rosario - R00640880

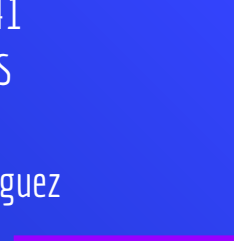
Kivan M. Lopez Raices - R00642441

COMP3600.COMPUTER GRAPHICS

Seccion:71316

Profesora: Elizabeth Maldonado Rodriguez

6 de mayo de 2026



Objetivos:

- Conocer el origen y la historia de Ursina Engine.
- Identificar sus características, ventajas y desventajas.
- Aprender a crear primitivos 3D (Box, Sphere, Cylinder, Cone).
- Aplicar colores, texturas, imágenes, video y audio a objetos.
- Implementar traslación y rotación en el espacio 3D.
- Explorar un mundo virtual creado con Ursina Engine.

Historia de Ursina Engine

- Ursina Engine es un motor de videojuegos y herramienta modelado 3D desarrollado por Petter Amland (conocido como pokepetter).
- Fue creado en julio de 2017 y lanzado públicamente en 2020.
- Está construido sobre Panda3D, un robusto motor gráfico de código abierto originalmente desarrollado por Disney y Carnegie Mellon University.



Historia de Ursina Engine

- El objetivo principal de Ursina fue simplificar drásticamente el desarrollo de aplicaciones 3D en Python, eliminando la complejidad de motores tradicionales como Unity o Unreal Engine.
- Su diseño minimalista y orientado a Python lo convirtió rápidamente en una herramienta popular, especialmente entre principiantes y educadores, permitiéndoles crear juegos, simulaciones 3D, prototipos y proyectos de gráficos por computadora.

Características

KMLR

- Transformaciones geométricas: Traslación, rotación y escala con sintaxis simple.
- Hot reloading: Recarga automática de assets durante el desarrollo.
- Editor integrado: Incluye un editor visual básico para manipulación de objetos.
- Basado en Python puro: No requiere conocer lenguajes adicionales; toda la lógica se escribe en Python 3.
- Basado en Entidades (Entity): Todo objeto en la escena es un Entity (similar a GameObject en Unity). Puede tener modelo, textura, color, posición, rotación, escala, video, audio y más.
- Sintaxis minimalista: Crear objetos 3D requiere una sola línea de código.

Características

KMLR

- ❖ Construido sobre Panda3D: Aprovecha un motor gráfico probado y maduro.
- ❖ Soporte de primitivos 3D: Box, Sphere, Cylinder, Cone, Plane, Quad y más.
- ❖ Sistema de texturas e iluminación integrado: Soporta imágenes y videos como texturas.
- ❖ Audio integrado: Reproducción de archivos de audio directamente en primitivos o la escena.

Características

KMLR

- ❖ Ciclo de vida integrado: Proporciona funciones automáticas como `update()` (se ejecuta cada frame) y un `input(key)` (responde a teclas).
- ❖ Prefabricados (Prefabs): Incluye controladores listos para usar como `FirstPersonController`, sistemas de partículas, grabador de pantalla, entre otros, ahorrando mucho tiempo de desarrollo.
- ❖ Multiplataforma (Cross-platform): Funciona en Windows, macOS y Linux.

Ventajas

EDFR

1. Curva de aprendizaje muy baja para programadores Python e ideal para proyectos universitarios y principiantes.
2. Código extremadamente conciso comparado con otros motores.
3. Ideal para prototipos, proyectos educativos y game jams.
4. Rápido prototipado donde se puede ver un objeto 3D en pantalla con 3 líneas de código.
5. Código 100% Python aprovechando toda su sintaxis y librerías.
6. Excelente para educación con su filosofía “pythonica” lo hace perfecto para enseñar programación y conceptos 3D.

Ventajas

- 7. Curva de aprendizaje muy baja para programadores Python e ideal para Gratuitos y de código abierto (licencia MIT).
- 8. Comunidad activa y documentación clara en línea.
- 9. Instalación sencilla con `pip install ursina`.

Desventajas

1. Rendimiento limitado para proyectos AAA, de gran escala, muy complejos o con miles de objetos, el rendimiento no es óptimo comparado con C++ o C#.
2. Hay menos documentación avanzada y, aunque es buena, no es tan extensa o profunda como la de Unity o Unreal.
3. El editor visual es básico y no tan completo como otros motores.
4. Limitado soporte para física avanzada sin librerías adicionales.
5. Menor soporte comercial al ser un proyecto de comunidad.
6. Es una comunidad pequeña, no tan grande como la de Unity, por lo que encontrar soluciones a problemas muy específicos puede ser más difícil.
7. Al ser un wrapper de Panda3D, Ursina hereda algunas de sus limitaciones, por lo que a veces puede ser necesario modificar el código de Panda3D.

Box (Cube) - Ancho, Alto y Profundida

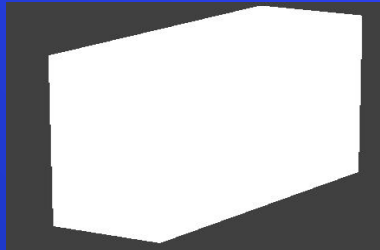
- El primitivo Box (cubo/caja) es la figura geométrica más básica en Ursina.
- Se crea usando la clase Entity con el modelo 'cube'.
- El parámetro scale recibe (x, y, z) donde x representa el ancho, y la altura y z la profundidad.
- Sintaxis: `Entity(model='forma', color=color, scale=(x, y, z))`

Codigo del Box

```
from ursina import *
app = Ursina()

# creacion del box con dimensiones (1x1x1)
# la sintaxis scale se usa como x y y z
# lo que significa que va a tener 1 de ancho
# 2 de altura y 3 de profundidad
Entity(model='Cube', color=color.white, scale=(1,2,3))

#permite el movimiento de la camara
EditorCamera()
app.run()
```



Sphere - Ancho, Alto y Profundidad

KMLR

- El primitivo Sphere (esfera) se crea usando el modelo 'sphere'.
- Aunque es simétrico por naturaleza, se puede deformar usando scale para crear elipsoides.
- Para una esfera perfecta se usa $\text{scale}=(r, r, r)$ o simplemente $\text{scale}=r$. Valores distintos crean un elipsoide.
- Sintaxis: `Entity(model='forma', color=color, scale=(x, y, z))`

Codigo con El Sphere

```
from ursina import *
app = Ursina()

#creacion de la esfera
# el scale hace que el ancho sea 1, altura sea 1 y profundidad sea 3
Entity(model='sphere', color=color.red, scale=(1,1,3))

#permite el movimiento de la camara
EditorCamera()
app.run()
```



Cilindro (Cylinder) - Ancho, Alto y Profundidad

- El cilindro en Ursina se crea usando la clase `Cylinder()`, la cual acepta los parámetros `height` para controlar la altura y `radius` para controlar el radio de la base.
- El eje vertical es Y, por lo que `height` define qué tan alto será el cilindro, mientras que `radius` define el tamaño del radio de forma uniforme en los ejes X y Z.
- Sintaxis: `Entity(model=Forma(height=Y, radius=X y Z), color=color)`

Código con figura:

```
from ursina import *  
app = Ursina()  
  
# creacion de el cylinder  
# se usa height y radius para que tenga una altura y una profundidad  
# definida porque sino no va a aparecer  
Entity(model=Cylinder(height=2, radius=0.5), color=color.blue)  
  
#permite el movimiento de la camara  
EditorCamera()  
app.run()
```

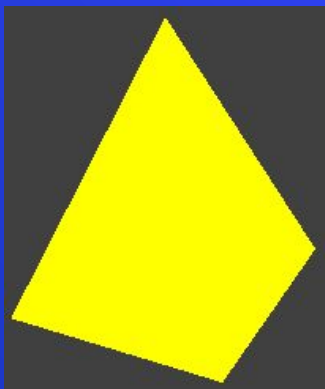


Cono (Cone) - Ancho, Alto y Profundidad

- El cono en Ursina se crea usando la clase `Cone()`, la cual acepta como parámetro el número de resolución (cantidad de lados o segmentos que forman la base circular).
- Un valor mayor como `Cone(16)` produce una base más suave y circular, mientras que valores menores como `Cone(4)` producen una base con forma de polígono.
- El eje vertical es Y, por lo que mientras más lados tenga el cono, más se aproxima a una figura cónica perfecta.
- Sintaxis: `Entity(model=Forma(), color=color, position(x, y, z), scale=r)`

Figura con código:

```
from ursina import *  
app = Ursina()  
  
# creacion de el cylinder  
# se usa scale para que el modelo aparezca  
Entity(model=Cone(), color=color.yellow, position=(0, 0, 0), scale=2)  
  
#permite el movimiento de la camara  
EditorCamera()  
app.run()
```



Color

- Ursina provee un módulo de color integrado con colores predefinidos y soporte para RGBA (Red, Green, Blue, Alpha). El color se asigna directamente al parámetro color Entity.
1. Sintaxis básica: `color=color.red`
 2. Sintaxis RGBA (0-1): `color=color.rgb(0, 0, 1, 0.5)` # azul
 3. Sintaxis hexadecimales: `color=color.hex('#FF5733')` # rojo-crimson peak
 4. Colores predefinidos disponibles: `color.red`, `color.green`, `color.blue`, `color.yellow`, `color.orange`, `color.white`, `color.black`, `color.gray`, `color.pink`, `color.violet`, `color.cyan`, `color.magenta`, `color.brown`, `color.gold`, `color.light_gray`, `color.dark_gray`, `color.lime`

Ejemplo de las figuras con color:

```
# 1. box con color
Entity(model='cube', color=color.red, scale=(1, 1, 1), position=(-3, 0, 0))

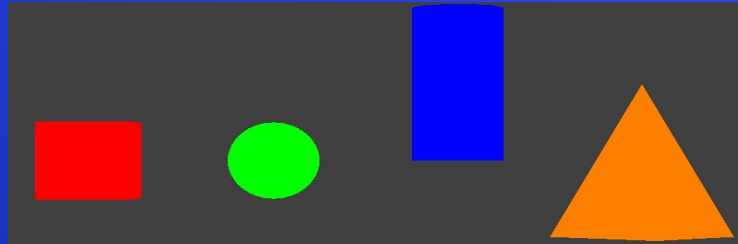
# 2. Esfera con color
Entity(model='sphere', color=color.green, position=(-1, 0, 0))

# 3. Cilindro con color
Entity(model=Cylinder(height=2, radius= 0.5), color=color.blue, position=(1, 0, 0))

# 4. Cono con color
Entity(model=Cone(), color=color.orange, scale=2, position=(3, 0, 0))

EditorCamera()

app.run()
```



Imagen

EDFR

- Ursina soporta la aplicación de imágenes como texturas en cualquier primitivo usando el parámetro texture.
- La imagen puede ser una URL, un path local o un archivo en la carpeta del proyecto. Formatos soportados: PNG, JPG, BMP, TGA, entre otros.
- Las texturas deben estar en la misma carpeta del proyecto o en una subcarpeta accesible.
- Sintaxis: `Entity(model='forma', texture="nombre", escala=(x, y, z))`

Codigo ejemplar de Imagen

```
from ursina import *  
app = Ursina()  
  
# para toda textura se le saca el color y se le pone la sintaxis  
# "texture="tu_imagen.jpg"  
Entity(model='sphere', texture="Mario-Sun.jpg", scale=(1,1,1))  
  
#permite el movimiento de la camara  
EditorCamera()  
app.run()
```



Video y Audio

- **Textura de Video:** Ursina permite usar videos como texturas dinámicas en primitivos usando MovieTexture de Panda3D, que es la librería base de Ursina. Formatos de video soportados: MP4. Sintaxis de Video: `Entity(model='forma', texture="nombre", escala=(x, y, z))`
- **Textura de Audio:** Ursina incluye soporte integrado de audio a través de la clase Audio. Se pueden reproducir sonidos y música en la escena o asociados a entidades. Formatos de audio soportados: MP3, WAV, OGG. El audio puede ser posicional (3D) o global (2D). Sintaxis de Audio: `Audio('cancion.mp3'), loop=True, autoplay=True)`

Ejemplo de video:

```
from ursina import *
app = Ursina()

# se utiliza la sintaxis texture="video.mp4"
# para ponerle video adentro del cuadro
Entity(model='Cube', texture='car.mp4', scale=(5,5,0.1))

#permite el movimiento de la camara
EditorCamera()
app.run()
```

[video ejemplar](#)

Ejemplo de audio:

```
from ursina import *
app = Ursina()

# se utiliza la sintaxis texture="video.mp4" para ponerle video adentro del cuadro
Entity(model='Cube', texture='mario 1-1.mp4', scale=(5,5,0.1))

# esto se utiliza para el audio, lo definimos como variable para evitar el delay
musica = Audio('mario 1-1.wav', loop=True, autoplay=True)

# esto se utiliza para darle delay apra que este on sync

def delayed_audio():
    musica.play()

invoke(delayed_audio, delay=0.6)

#permite el movimiento de la camara
EditorCamera()
app.run()
```

[Video con audio](#)

Traslación (Position)

KMLR

- La traslación en Ursina mueve un objeto en el espacio 3D usando el parámetro `position` o los atributos `x`, `y`, `z` de la entidad.
- `time.dt` (delta time) garantiza movimiento independiente de la velocidad del procesador.
- `animate_position()` crea una animación suave hacia una posición destino.
- Sintaxis: `Entity(model='forma', texture="nombre", escala=(x, y, z), position=(x, y, z))`

Ejemplo de Traslación

cam1_primitives > translation.py

```
1  from ursina import *
2  app = Ursina()
3
4  # se incluye position en el programa para cambiar las positions de los modelos
5  Entity(model='sphere', texture="Mario-Sun.jpg", scale=(1,1,1), position=(3,4,1), rotation=(0,90,0))
6
7  # cylinde rpara `tronco`
8  Entity(model=Cylinder(height=2, radius= 0.5), texture='pine.jpg', position=(1, 0, 1))
9
10 #hojas de arbol con la sphere
11 Entity(model='sphere', texture="leaves.jpg", scale=(2,2,2), position=(1,2,1))
12
13 #permite el movimiento de la camara
14 EditorCamera()
15 app.run()
```



Rotación

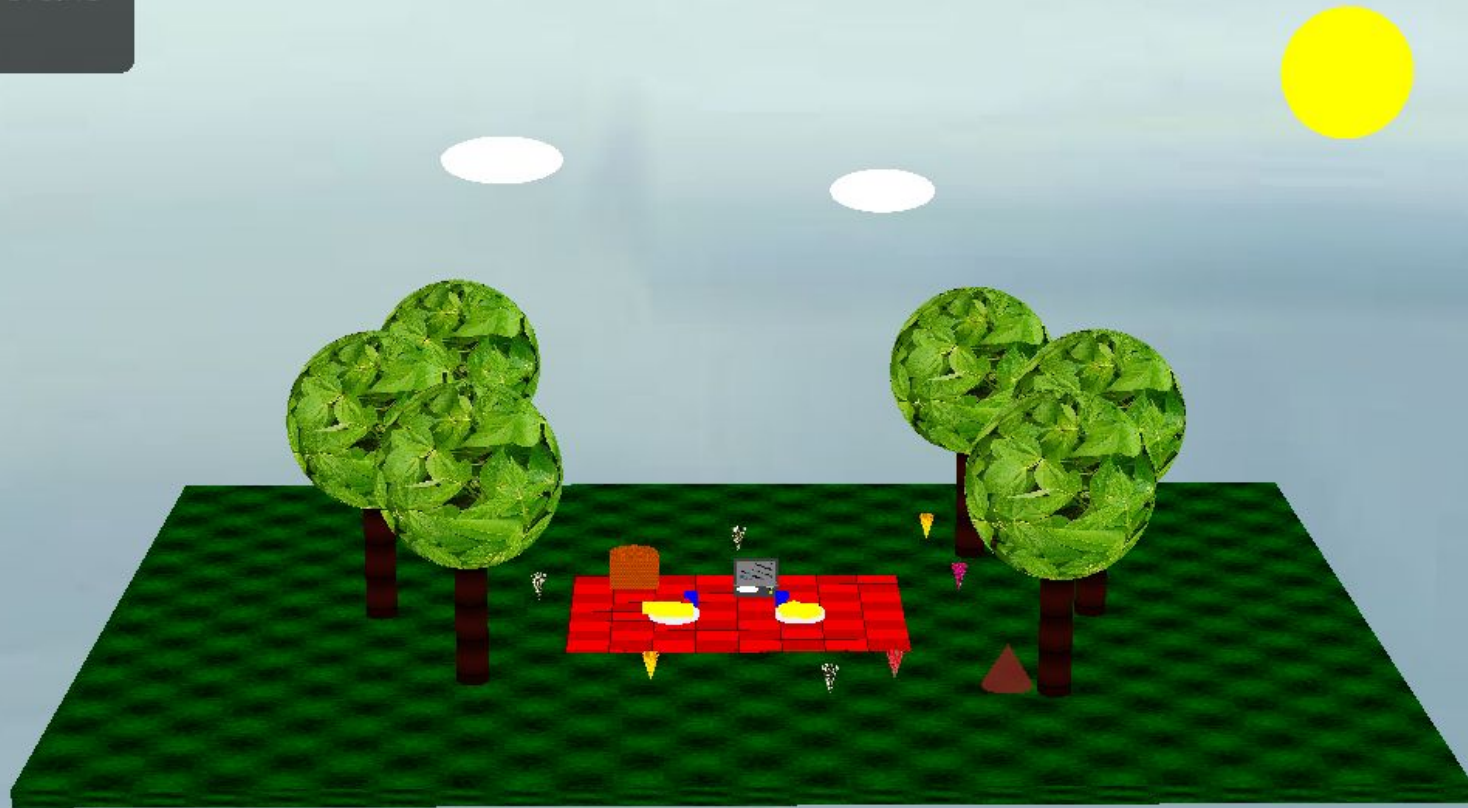
- La rotación en Ursina se controla con el parámetro `rotation` o los atributos `rotation_x`, `rotation_y`, `rotation_z`.
- Los valores están en grados (0-360). `rotate()` es diferente a `rotation`: `rotation` fija el ángulo absoluto; la suma incremental en `update()` crea rotación continua. `animate_rotation()` crea una animación suave.
- Sintaxis: `Entity(model='forma', texture="nombre", escala=(x, y, z), position=(x, y, z), rotation=(x, y, z))`

Ejemplo de rotación:

```
exam1_primitives > rotation.py
1  from ursina import *
2  app = Ursina()
3
4  # se incluye rotation en el programa para mover la imagen
5  Entity(model='sphere',texture="Mario-Sun.jpg", scale=(1,1,1), position=(1,0,1), rotation=(0,90,0))
6  #imagen a 180
7  Entity(model='sphere',texture="Mario-Sun.jpg", scale=(1,1,1), position=(3,0,1), rotation=(0,180,0))
8  # imagen a 270
9  Entity(model='sphere',texture="Mario-Sun.jpg", scale=(1,1,1), position=(-1,0,1), rotation=(0,270,0))
10 #imagen a 0
11 Entity(model='sphere',texture="Mario-Sun.jpg", scale=(1,1,1), position=(-3,0,1), rotation=(0,0,0))
12
13 #permite el movimiento de la camara
14 EditorCamera()
15 app.run()
```



COMP 3600 – Picnic Set | Ursina
Q=salir M=mute/unmute audio



KSPR KMLR **Video del Mundo Virtual**

EDFR

<https://youtu.be/3xyZPNlxCD4>



Código:

```
picnic_ursina.py > ...
1  from ursina import *
2  import os
3  from panda3d.core import MovieTexture # ← FIX 1: Added MovieTexture import
4
5  app = Ursina(title='COMP 3600 | Picnic Set 3D | Ursina', borderless=False)
6  EditorCamera()
7
8  ASSET_DIR = os.path.dirname(os.path.abspath(__file__))
9  def asset(name):
10     return os.path.join(ASSET_DIR, name)
11
12  # Colors
13  C_GRASS = color.green
14  C_BLANKET = color.red
15  C_BASKET = color.orange
16  C_FOOD = color.yellow
17  C_PLATE = color.white
18  C_CUP = color.blue
19  C_BARK = color.brown
20  C_LEAVES = color.lime
21  C_APPLE = color.red
22  C_SUN = color.yellow
23  C_FLOWER = color.magenta + color.pink
24  C_CLOUD = color.white
25  C_Dirt = color.brown * 0.8
26
27  # DVD player colors
28  C_DVD_BODY = color.dark_gray
29  C_DVD_SPK = color.black
30  C_DVD_SCREEN = color.black
31  C_DVD_BEZEL = color.gray
32  C_DVD_BTN = color.light_gray
33  C_DVD_LED = color.lime
34  C_DVD_DISK = color.gray + color.white * 0.5
```

Código:

```
36 # VIDEO TEXTURE - Load video for DVD screen
37
38 video_texture = None
39 video_playing = False
40 video_path = asset('car.mp4') # Make sure 'car.mp4' is in your project folder
41
42 if os.path.exists(video_path):
43     try:
44         video_texture = MovieTexture(video_path)
45         print(f"✓ Video loaded: {video_path}")
46         try:
47             print(f"    Duration: {video_texture.get_video_length()} seconds")
48         except:
49             pass
50     except Exception as e:
51         print(f"X Error loading video: {e}")
52         video_texture = None
53 else:
54     print(f"X Video file not found: {video_path}")
55     print("    Make sure 'car.mp4' is in the same folder as your script")
56
57 # BOX
58
59 # Ground
60 ground = Entity(
61     model='cube',
62     texture='grass.png',
63     color=C_GRASS,
64     position=(0, -0.5, 0),
65     scale=(30, 0.4, 24),
66 )
```

Código:

```
67 # Blanket
68 blanket = Entity(
69     model='cube',
70     texture='blanket.png',
71     color=C_BLANKET,
72     position=(0, -0.28, 0),
73     scale=(8, 0.08, 6),
74 )
75
76 # Basket
77 basket_body = Entity(
78     model='cube',
79     texture='wicker.png',
80     color=C_BASKET,
81     position=(-2.5, 0.1, 2.2),
82     scale=(1.2, 0.7, 0.8),
83 )
84
85 # Sandwiches
86 sandwich = Entity(
87     model='cube',
88     color=C_FOOD,
89     position=(-1.5, 0, -0.7),
90     scale=(0.9, 0.16, 0.55),
91 )
92 sandwich2 = Entity(
93     model='cube',
94     color=C_FOOD,
95     position=(1.5, 0, -0.7),
96     scale=(0.9, 0.16, 0.55),
97 )
98
99 print("✓ BOX: ground, blanket, basket, sandwiches")
100
101 # SPHERE
102
103 # Tree canopies - far back corners to frame the scene
104 tree_canopy = Entity(model='sphere', texture='leaves.jpg', position=( 7, 4.5, -6), scale=4.2)
105 tree_canopy2 = Entity(model='sphere', texture='leaves.jpg', position=(-7, 4.5, 6), scale=4.2)
106 tree_canopy3 = Entity(model='sphere', texture='leaves.jpg', position=( 6, 4.5, 5), scale=4.2)
107 tree_canopy4 = Entity(model='sphere', texture='leaves.jpg', position=(-6, 4.5, -5), scale=4.2)
108 tree_canopy5 = Entity(model='sphere', texture='leaves.jpg', position=( 8.5, 4.5, 0), scale=4.2)
109 tree_canopy6 = Entity(model='sphere', texture='leaves.jpg', position=(-8.5, 4.5, 0), scale=4.2)
110
111 # Basket lid - sits on top of basket (basket top = 0.3 + 0.45 = 0.75)
112 basket_lid = Entity(
113     model='sphere',
114     texture='wicker.png',
115     color=C_BASKET,
116     position=(-2.5, 0.6, 2.2),
117     scale=(1.2, 0.3, 0.8),
118 )
119
120 # Apples - on top of the blanket surface
121 apple1 = Entity(model='sphere', color=C_APPLE, position=(-1.8, 0, -0.2), scale=0.22)
122 apple2 = Entity(model='sphere', color=C_APPLE, position=( 1.8, 0, -0.3), scale=0.22)
123 # Apple on the blanket (near basket)
124 apple3 = Entity(model='sphere', color=C_APPLE, position=(1.3, 0, -0.1), scale=0.22)
125
126 # Sun - high in sky, off to one corner
127 sun = Entity(model='sphere', color=C_SUN, position=(12, 14, -10), scale=2.6)
128
129 # Clouds - high and distant, behind and above the scene
130 cloud1 = Entity(model='sphere', color=C_CLOUD, position=(-6, 9, 10), scale=(3.2, 1.2, 2.2))
131 cloud2 = Entity(model='sphere', color=C_CLOUD, position=( 4, 8, 11), scale=(2.8, 1.1, 2.0))
132
```

Código:

```
134 # DVD player - power LED (sphere) on top-right corner of the unit
135 dvd_led = Entity(
136     model = 'sphere',
137     color = C_DVD_LED,
138     position = (0.85, -0.13, 1.20),
139     scale = 0.08,
140 )
141
142 print("✓ SPHERE: trees, apples, sun, clouds, basket lid, DVD LED")
143
144 # CYLINDER
145
146 # Tree trunks
147 tree_trunk = Entity(
148     model = Cylinder(resolution=10, radius=0.5, height=2),
149     texture = 'bark.png',
150     color = C_BARK,
151     position = (7, -0.3, -6),
152     scale = (0.75, 2.6, 0.75),
153 )
154 tree_trunk2 = Entity(
155     model = Cylinder(resolution=10, radius=0.5, height=2),
156     texture = 'bark.png',
157     color = C_BARK,
158     position = (-7, -0.3, 6),
159     scale = (0.75, 2.6, 0.75),
160 )
161 tree_trunk3 = Entity(
162     model = Cylinder(resolution=10, radius=0.5, height=2),
163     texture = 'bark.png',
164     color = C_BARK,
165     position = (6, -0.3, 5),
166     scale = (0.75, 2.6, 0.75),
167 )
```

```
168 tree_trunk4 = Entity(
169     model = Cylinder(resolution=10, radius=0.5, height=2),
170     texture = 'bark.png',
171     color = C_BARK,
172     position = (-6, -0.3, -5),
173     scale = (0.75, 2.6, 0.75),
174 )
175 tree_trunk5 = Entity(
176     model = Cylinder(resolution=10, radius=0.5, height=2),
177     texture = 'bark.png',
178     color = C_BARK,
179     position = (8.5, -0.3, 0),
180     scale = (0.75, 2.6, 0.75),
181 )
182 tree_trunk6 = Entity(
183     model = Cylinder(resolution=10, radius=0.5, height=2),
184     texture = 'bark.png',
185     color = C_BARK,
186     position = (-8.5, -0.3, 0),
187     scale = (0.75, 2.6, 0.75),
188 )
```

Código:

```
190 # Plates
191 plate1 = Entity(model=Cylinder(20, 0.4, 0.5), texture='plate.png', color=C_PLATE, position=(-1.5, -0.30, -0.5),
192 scale=(1.5, 0.15, 1.5))
193 plate2 = Entity(model=Cylinder(20, 0.4, 0.5), texture='plate.png', color=C_PLATE, position=( 1.5, -0.30, -0.5),
194 scale=(1.5, 0.15, 1.5))
195
196 # Cups
197 cup1 = Entity(model=Cylinder(12, 0.5, 2), color=C_CUP, position=(-1.10, -0.88, 0.5), scale=(0.32, 0.32, 0.32))
198 cup2 = Entity(model=Cylinder(12, 0.5, 2), color=C_CUP, position=( 1.1, -0.88, 0.5), scale=(0.32, 0.32, 0.32))
199
200 # DVD player - cylindrical play/pause button on the side panel
201 dvd_btn_play = Entity(
202     model    = Cylinder(resolution=12, radius=1.5, height=0.5),
203     color    = C_DVD_BTN,
204     position = (0.85, -0.17, 1.69),
205     scale    = (0.07, 0.04, 0.07),
206 )
207 # DVD player - small circular speaker grille dots (3 cylinders)
208 dvd_spk1 = Entity(model=Cylinder(8, 0.5, 0.5), color=C_DVD_SPK, position=(0.82, -0.17, 1.33), scale=(0.035, 0.03, 0.035))
209 dvd_spk2 = Entity(model=Cylinder(8, 0.5, 0.5), color=C_DVD_SPK, position=(0.82, -0.17, 1.43), scale=(0.035, 0.03, 0.035))
210 dvd_spk3 = Entity(model=Cylinder(8, 0.5, 0.5), color=C_DVD_SPK, position=(0.82, -0.17, 1.50), scale=(0.035, 0.03, 0.035))
```

Código:

```
211
212 # DVD disc tray slot - thin flat cylinder laying flat on top of the base
213 dvd_tray = Entity(
214     model      = Cylinder(resolution=20, radius=0.5, height=0.1),
215     color      = C_DVD_DISK,
216     position   = (0.3, -0.148, 1.5),
217     rotation   = (0, 0, 0),
218     scale      = (0.55, 0.04, 0.55),
219 )
220
221 print("✓ CYLINDER: tree trunks, plates, cups, DVD buttons, speaker grille, tray")
222
223 # CONE
224
225 # Anthill - on the ground, off the blanket
226 anthill = Entity(model=Cone(16), color=C_Dirt, position=(6.0, 0.19, -5.5), scale=(1.2, 1.0, 1.2))
227
228 # Flowers - in the grass on opposite corners of the scene
229 flower1 = Entity(model=Cone(8), texture='sunflower.jpg', position=(-2.0, 0, -5.0), scale=(0.4, 0.65, 0.4), rotation=(180, 0, 0))
230 flower2 = Entity(model=Cone(8), texture='sunflower.jpg', position=( 5.0, 0,  6.5), scale=(0.4, 0.65, 0.4), rotation=(180, 0, 0))
```

Código:

```
231 flower3 = Entity(model=Cone(8), texture='sunflower.jpg', position=(-5.5, 0, 5.0), scale=(0.4, 0.65, 0.4), rotation=(180, 0, 0))
232 flower4 = Entity(model=Cone(8), texture='daisies.jpg', position=( 2.0, 0, -6.0), scale=(0.4, 0.65, 0.4), rotation=(180, 0, 0))
233 flower5 = Entity(model=Cone(8), texture='daisies.jpg', position=( 0.10, 0, 5.3), scale=(0.4, 0.65, 0.4), rotation=(180, 0, 0))
234 flower6 = Entity(model=Cone(8), texture='tulip.jpg', position=( 3.5, 0, -5.0), scale=(0.4, 0.65, 0.4), rotation=(180, 0, 0))
235 flower7 = Entity(model=Cone(8), texture='roses.jpg', position=(-6.0, 0, -3.0), scale=(0.4, 0.65, 0.4), rotation=(180, 0, 0))
236 flower8 = Entity(model=Cone(8), texture='roses.jpg', position=( 5.5, 0, 2.0), scale=(0.4, 0.65, 0.4), rotation=(180, 0, 0))
237 flower9 = Entity(model=Cone(8), texture='daisies.jpg', position=(-4.8, 0, 1.2), scale=(0.4, 0.65, 0.4), rotation=(180, 0, 0))
238 flower10 = Entity(model=Cone(8), texture='tulip.jpg', position=( -6.0, 0, 4.0), scale=(0.4, 0.65, 0.4), rotation=(180, 0, 0))
239
240 print("✓ CONE: anthill, flowers")
241
242 # PORTABLE DVD PLAYER
243
244 # Base / bottom half of the clamshell unit
245 dvd_base = Entity(
246     model = 'cube',
247     color = C_DVD_BODY,
248     position = (0.5, -0.195, 1.5),
249     scale = (1.1, 0.08, 0.75),
250 )
251
252 # Lid / top half
253 dvd_lid = Entity(
254     model = 'cube',
255     color = C_DVD_BODY,
256     position = (0.5, 0.10, 2.1),
257     rotation = (-45, 0, 0),
258     scale = (1.1, 0.06, 0.75),
259 )
260
261 # Screen bezel
262 dvd_bezel = Entity(
263     model = 'cube',
264     color = C_DVD_BEZEL,
265     position = (0.5, 0.13, 2.0),
266     rotation = (-45, 0, 0),
267     scale = (0.98, 0.065, 0.62),
268 )
269
270 # Screen surface with video texture
271 if video_texture:
272     dvd_screen = Entity(
273         model = 'cube',
274         texture = 'car.mp4',
275         color = color.white,
276         position = (0.5, 0.13, 2.0),
277         rotation = (-45, 0, 0),
278         scale = (0.82, 0.067, 0.48)
```

Código:

```
279     )
280     print("✓ Video screen created with texture")
281 else:
282     dvd_screen = Entity(
283         model      = 'cube',
284         color      = color.black,
285         position   = (0.5, 0.13, 2.0),
286         rotation   = (-45, 0, 0),
287         scale      = (0.82, 0.067, 0.48),
288     )
289     print("X No video texture - screen is black")
290
291 # Hinge detail
292 dvd_hinge = Entity(
293     model      = 'cube',
294     color      = color.gray,
295     position   = (0.5, -0.155, 1.84),
296     scale      = (1.12, 0.04, 0.06),
297 )
298
```

Código:

```
299 print("✓ DVD PLAYER: base, lid, screen bezel, VIDEO screen, hinge, button, speaker, tray, LED")
300
301 # AUDIO
302
303 # Debug - add this before the Audio line:
304 audio_file = 'nature.wav' # or 'nature.mp3'
305 print(f"Current folder: {os.getcwd()}")
306 print(f"Looking for: {audio_file}")
307 print(f"File exists: {os.path.exists(audio_file)}")
308
309 ambient_audio = Audio('nature.wav', loop=True, autoplay=True, volume=0.55)
310
311 # TRASLACION
312
313 sandwich_moved = Entity(
314     model    = 'cube',
315     color    = C_FOOD,
316     position = sandwich.position + Vec3(-0.3, 0.1, -0.1),
317     scale    = sandwich.scale,
318 )
319 # Apple translated onto plate1
```

Código:

```
320 apple_on_plate = Entity(  
321     model    = 'sphere',  
322     color    = C_APPLE,  
323     position = plate1.position + Vec3(0.3, 0.32, 0.3),  
324     scale    = 0.22,  
325 )  
326 print("✓ TRASLACION: sandwich, apple on plate")  
327  
328 # ROTACION  
329  
330 sandwich_rotated = Entity(  
331     model    = 'cube',  
332     color    = C_FOOD,  
333     position = (1.5, 0, -0.7),  
334     scale    = (0.9, 0.16, 0.55),  
335     rotation = (0, 45, 0),  
336 )  
337 print("✓ ROTACION: basket 25°/Y, sandwich 45°/Y")  
338  
339 # SKY
```

Código:

```
340
341 sky = sky()
342
343 # Lighting
344 AmbientLight(color=color.rgb(180, 190, 200, 255))
345 DirectionalLight(y=10, z=-5, shadows=True, rotation=(45, -45, 0))
346
347 # HUD
348 Text(
349     text = ("<white>COMP 3600 [ ] Picnic Set | Ursina\n"
350             "<scale:0.8>Q=salir M=mute/unmute audio"),
351     position = (-0.85, 0.47),
352     color = color.white,
353     background = True,
354 )
355
356 # CONTROLES
357
358 muted = False
359
```

Código:

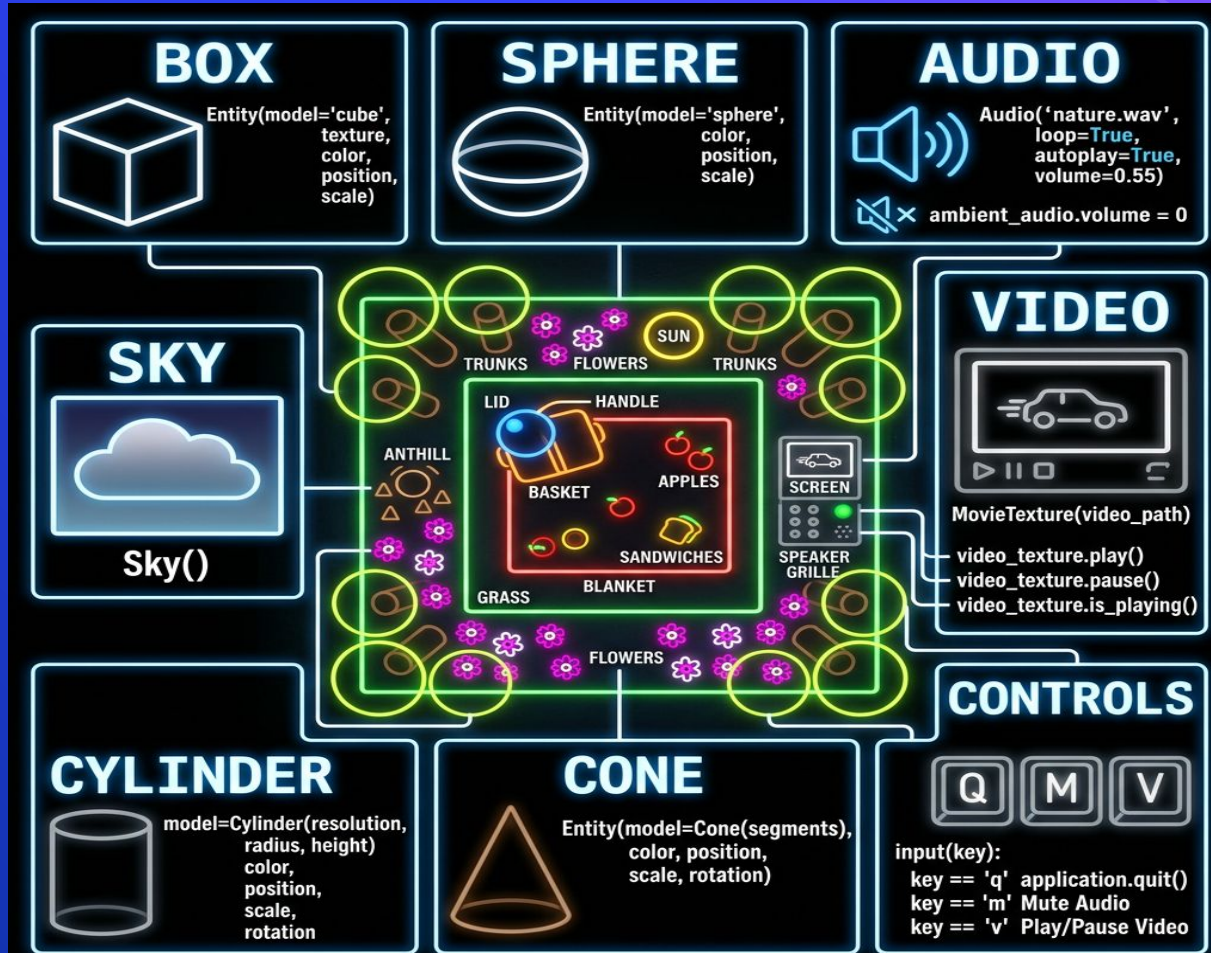
```
360 def input(key):
361     global muted
362     if key == 'q':
363         application.quit()
364     elif key == 'm':
365         muted = not muted
366         if ambient_audio:
367             ambient_audio.volume = 0 if muted else 0.55
368         print("Audio: MUTED" if muted else "Audio: ON")
369
370 # Start video automatically if loaded
371 if video_texture:
372     try:
373         video_texture.play()
374         print("✓ Video started automatically")
375     except Exception as e:
376         print(f"Could not auto-start video: {e}")
377
378 app.run()
```

Zip Mundo Virtual

`pip install ursina`

<https://drive.google.com/file/d/1pl6nr-T3QXpdILt-TaMf0PuUybv3YysG/view?usp=sharing>

Infografía



Referencias a URLs

Páginas Web Oficiales

Amland, P. [pokepetter]. (2020). *Ursina engine: A Python powered, open source game engine.*

<https://www.ursinaengine.org/>

Amland, P. [pokepetter]. (2020). *Ursina engine – API reference.*

https://www.ursinaengine.org/api_reference.html

Amland, P. [pokepetter]. (2020). *Ursina [Paquete de software].* PyPI. <https://pypi.org/project/ursina/>

Amland, P. [pokepetter]. (2020). *pokepetter/ursina [Repositorio de software].* GitHub.

<https://github.com/pokepetter/ursina>

Carnegie Mellon University. (s.f.). *Introduction to Panda3D – Panda3D manual.*

<https://docs.panda3d.org/1.10/python/introduction/index>

Disney & Carnegie Mellon University. (2002). *Panda3D: Free open source 3D game and simulation engine.*

<https://www.panda3d.org/>

Referencias a URLs

Tutoriales en Video (YouTube), Páginas Web y App

Amland, P. [pokepetter]. (s.f.). Documentation. Ursina Engine. <https://www.ursinaengine.org/documentation.html>

Amland, P. [pokepetter]. (s.f.). Introduction tutorial. Ursina Engine. https://www.ursinaengine.org/introduction_tutorial.html

Amland, P. [pokepetter]. (s.f.). Inventory tutorial. Ursina Engine. https://www.ursinaengine.org/inventory_tutorial.html

Amland, P. [pokepetter]. (s.f.). Platformer tutorial. Ursina Engine. https://www.ursinaengine.org/platformer_tutorial.html

Amland, P. [pokepetter]. (s.f.). Ursina game engine tutorial for dummies. Ursina Engine.
https://www.ursinaengine.org/ursina_for_dummies.html

Clear Code. (2021, enero 22). Introduction to Ursina (by coding the Breakout Game) [Video]. YouTube.
<https://www.youtube.com/watch?v=P8j6K6p-lzY>

CloudBit Ltd. (2026). Python coding editor & IDE app (Versión actual) [Aplicación móvil]. Google Play Store.
https://play.google.com/store/apps/details?id=com.markodevcic.python_code_pad&hl=en

DaFluffyPotato. (s.f.). DaFluffyPotato [Canal de YouTube]. <https://www.youtube.com/@DaFluffyPotato>

Ursina Tutorials. (s.f.). Ursina tutorials [Canal de YouTube]. <https://www.youtube.com/@ursinatutorials>

Referencias a URLs

Cursos MOOC

Coursera. (s.f.). Python for everybody specialization. <https://www.coursera.org>

freeCodeCamp. (s.f.). Scientific computing with Python. <https://www.freecodecamp.org>

Documentación y Manuales

Amland, P. [pokepetter]. (s.f.). Animation. Ursina Engine. <https://www.ursinaengine.org/animation.html>

Amland, P. [pokepetter]. (s.f.). Coordinate system. Ursina Engine.
https://www.ursinaengine.org/coordinate_system.html

Amland, P. [pokepetter]. (s.f.). Documentation. Ursina Engine. <https://www.ursinaengine.org/documentation.html>

Amland, P. [pokepetter]. (s.f.). Entity basics. Ursina Engine. https://www.ursinaengine.org/entity_basics.html

Blender Foundation. (2026). Blender 5.1 – The free and open source 3D creation software. <https://www.blender.org/>

Carnegie Mellon University & Disney. (s.f.). Panda3D manual. <https://docs.panda3d.org/>

Referencias a URLs

Herramientas y Entornos de Desarrollo

JetBrains. (s.f.). *PyCharm: The Python IDE for data science and web development* [Software].

<https://www.jetbrains.com/pycharm/>

Microsoft. (s.f.). *Visual Studio Code (Versión actual)* [Software]. <https://code.visualstudio.com>

Comunidades

Amland, P. [pokepetter]. (s.f.). *Ursina engine – Discord server* [Servidor de Discord].

<https://discord.gg/ursina>

Stack Overflow. (s.f.). Questions tagged [ursina]. <https://stackoverflow.com/questions/tagged/ursina>